```
    void add_or_update_mapping(Key const& key,Value const& value)
    {
        std::unique_lock<boost::shared_mutex> lock(mutex);          ◀-4
        bucket_iterator const found_entry=find_entry_for(key);
        if(found_entry==data.end())
        {
            data.push_back(bucket_value(key,value));
        }
        else
        {
            found_entry->second=value;
        }
    }

    void remove_mapping(Key const& key)
    {
        std::unique_lock<boost::shared_mutex> lock(mutex);          ◀-5
        bucket_iterator const found_entry=find_entry_for(key);
        if(found_entry!=data.end())
        {
            data.erase(found_entry);
        }
    }
};

std::vector<std::unique_ptr<bucket_type> > buckets;          ◀-6
Hash hasher;

bucket_type& get_bucket(Key const& key) const          ◀-7
{
    std::size_t const bucket_index=hasher(key)%buckets.size();
    return *buckets[bucket_index];
}
```